

AWS services explained

explain each service provide aws in detail with some flowchart and how it works and there client of each one take longer and make a good note to read and understand

Aws Services — Detailed Notes, Flowcharts & How They Work

AWS Services — Detailed Notes, Flowcharts & How They Work

Designed for learning and reference. Covers core AWS services by category, how each works, typical clients and real-world use-cases, and simple flowcharts/diagrams to visualise interactions.

Table of contents

1. Quick intro & AWS global infra
2. Compute
 - EC2, Lambda, ECS, EKS, Elastic Beanstalk
3. Storage
 - S3, EBS, EFS, Glacier (S3 Glacier)
4. Databases
 - RDS, Aurora, DynamoDB, ElastiCache, Redshift
5. Networking & Content Delivery
 - VPC, Route 53, ELB (ALB/NLB), CloudFront
6. Security & Identity
 - IAM, KMS, Cognito, Shield, WAF
7. DevOps & CI/CD
 - CodeCommit, CodeBuild, CodeDeploy, CodePipeline, CodeArtifact

8. Monitoring & Logging
 - CloudWatch, CloudTrail, X-Ray
 9. Analytics & Streaming
 - Kinesis, EMR, Athena, Glue
 10. Machine Learning & AI
 - SageMaker
 11. Management, Provisioning & Migration
 - CloudFormation, AWS Config, Systems Manager, DMS
 12. Serverless & Event-driven patterns
 - API Gateway, EventBridge, Step Functions
 13. IoT, Mobile & Edge
 - IoT Core, Amplify, Greengrass
 14. Typical architecture patterns (with flowcharts)
 - Classic web app (EC2 + RDS)
 - Serverless web app (API Gateway + Lambda)
 - Data pipeline (Kinesis/Glue/Redshift)
 15. Quick cheat-sheet: When to choose which service
 16. Study tips & further reading
-

1) Quick intro & AWS global infrastructure

What is AWS (brief): Amazon Web Services (AWS) is a cloud provider offering a large set of on-demand services for compute, storage, networking, databases, analytics, machine learning, security, management tools and more. These services are available in multiple geographic **Regions** and **Availability Zones (AZs)** to provide redundancy and low-latency for customers.

How AWS is organised: Services are grouped by category (Compute, Storage, Database, Networking, etc.). Regions contain multiple AZs — AZs are isolated data center groups for high availability.

2) Compute

Amazon EC2 (Elastic Compute Cloud)

- **What:** Virtual machines (VMs) in the cloud (instances) you can choose by CPU, memory, storage and network.
- **How it works (high level):** You pick an AMI (Amazon Machine Image) → choose instance type → launch in a VPC subnet → attach storage (EBS) → set up security groups. You manage OS, runtime, scaling (Auto Scaling Groups).
- **Typical clients / use-cases:** General-purpose servers, legacy applications, stateful services, custom OS requirements, batch processing.

Key concepts: AMI, instance types, EBS volumes, security groups, key pairs, Auto Scaling Groups, Elastic IPs.

AWS Lambda (serverless)

- **What:** Run code (functions) without provisioning servers. Triggers: API Gateway, S3 events, DynamoDB streams, EventBridge, etc.
- **How it works:** You write a function, deploy to Lambda. When an event occurs, AWS provisions execution environment (container), runs the function, then scales automatically. Billing is per 100ms of execution time (plus memory size).
- **Typical clients / use-cases:** Event-driven workloads, lightweight APIs, cron jobs, realtime file processing (images, logs), backend for mobile apps.

Amazon ECS & EKS (containers)

- **ECS:** AWS-managed container orchestration (tight AWS integration). You define tasks and services, AWS manages scheduling.
- **EKS:** Managed Kubernetes — AWS runs the control plane, you manage worker nodes or use Fargate for serverless containers.
- **Typical clients / use-cases:** Microservices, containerized applications, CI/CD pipelines.

Elastic Beanstalk

- **What:** PaaS for quickly deploying web apps. Upload code, EB handles provisioning (EC2, load balancer, autoscaling) and platform patches.
- **Use-case:** Fast prototyping, simple web apps where deep infra control isn't needed.

3) Storage

Amazon S3 (Simple Storage Service)

- **What:** Object storage for files (images, backups, logs, static website hosting).
- **How it works:** Data stored as objects in buckets. Highly durable and available. Access via REST APIs, SDKs, console. Features: lifecycle policies, versioning, encryption, replication.
- **Clients / use-cases:** Static assets for websites, data lake storage, backups, media stores.

EBS (Elastic Block Store)

- **What:** Block storage attached to EC2 instances (disk volumes).
- **Use-case:** OS disks, databases on EC2, applications requiring block-level storage.

EFS (Elastic File System)

- **What:** Managed NFS file system accessible by multiple EC2 instances simultaneously.
- **Use-case:** Shared file storage for apps, containers, big data frameworks.

S3 Glacier & Glacier Deep Archive

- **What:** Long-term archival storage at low cost, higher retrieval latency.
- **Use-case:** Compliance archives, cold backups.

4) Databases

Amazon RDS (Relational Database Service)

- **What:** Managed relational databases (MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, Amazon Aurora).
- **How it works:** AWS handles provisioning, patching, backups, Multi-AZ failover and snapshots.
- **Clients / use-cases:** OLTP applications, web app backends, SaaS platforms.

Amazon Aurora

- **What:** Cloud-optimized relational DB compatible with MySQL and PostgreSQL, better performance and scalability.
- **Use-case:** High-performance transactional databases.

Amazon DynamoDB

- **What:** Fully managed NoSQL key-value and document database with single-digit millisecond latency at any scale.
- **Use-case:** High throughput apps (gaming, ad tech), session stores, user profiles, IoT telemetry.

Amazon ElastiCache

- **What:** Managed in-memory data stores (Redis, Memcached).
- **Use-case:** Caching, session stores, leaderboards, real-time analytics.

Amazon Redshift

- **What:** Petabyte-scale data warehouse for analytics.
 - **Use-case:** BI, reporting, large-scale data analytics.
-

5) Networking & Content Delivery

Amazon VPC (Virtual Private Cloud)

- **What:** Isolation boundary for your AWS resources. You define subnets, route tables, NAT gateways, security groups and NACLs.
- **Use-case:** Secure networks for all AWS-hosted applications.

Route 53

- **What:** DNS service + health checks + domain registration + routing policies (weighted, latency, failover).
- **Use-case:** Domain name management and traffic routing.

Elastic Load Balancing (ALB, NLB)

- **ALB:** Layer 7 load balancing (HTTP/HTTPS) with path-based routing, host-based routing.
- **NLB:** High-performance Layer 4 load balancing for TCP/UDP.

Amazon CloudFront

- **What:** Global CDN that caches content at edge locations for low-latency delivery.
 - **Use-case:** Accelerate websites, APIs, media streaming.
-

6) Security & Identity

IAM (Identity and Access Management)

- **What:** Central authN/authZ for AWS accounts — users, groups, roles, policies.
- **Best practice:** Use least privilege, roles for services, avoid long-lived root credentials.

KMS (Key Management Service)

- **What:** Managed encryption keys (customer-managed or AWS-managed) for encrypting data at rest.

AWS Shield & WAF

- **Shield:** DDoS protection (Standard is automatic, Advanced is paid).
- **WAF:** Web application firewall to filter malicious requests.

Cognito

- **What:** Managed identity provider for mobile & web apps — user pools, federated sign-in (Google, Facebook, SAML).
-

7) DevOps & CI/CD

- **CodeCommit:** Git repositories in AWS.
- **CodeBuild:** Managed build service.

- **CodeDeploy:** Deploys to EC2, Lambda, on-prem.
- **CodePipeline:** Orchestrates full CI/CD pipelines.
- **CodeArtifact:** Package repository for Maven, npm, pip.

Pattern: Push → CodeCommit → CodePipeline triggers CodeBuild → Tests → CodeDeploy to EC2/ECS/Lambda.

8) Monitoring & Logging

CloudWatch

- **What:** Metrics, logs, alarms, dashboards. Used to monitor AWS resources and applications.

CloudTrail

- **What:** Audit log of all API calls made in your account — required for security/compliance tracking.

AWS X-Ray

- **What:** Distributed tracing for microservices and serverless apps to identify latency and errors.
-

9) Analytics & Streaming

Kinesis

- **What:** Real-time streaming data platform (Kinesis Data Streams, Data Firehose, Data Analytics).
- **Use-case:** Clickstream analytics, log collection, streaming ETL.

EMR (Elastic MapReduce)

- **What:** Managed Hadoop/Spark clusters for big data processing.

Athena

- **What:** Serverless interactive query service for data in S3 using SQL.

Glue

- **What:** Managed ETL and data cataloging service.
-

10) Machine Learning & AI

SageMaker

- **What:** End-to-end managed service for building, training, and deploying ML models (notebooks, training jobs, endpoints, model monitoring).
 - **Use-case:** Predictive analytics, recommendation engines, image/text ML workloads.
-

11) Management, Provisioning & Migration

CloudFormation

- **What:** Infrastructure as Code — declare templates (YAML/JSON) to provision AWS resources reproducibly.

AWS Config

- **What:** Tracks configuration changes and assesses configuration compliance.

Systems Manager

- **What:** Centralized operational tooling (Run Command, Patch Manager, Parameter Store).

DMS (Database Migration Service)

- **What:** Migrate databases to AWS with minimal downtime.
-

12) Serverless & Event-driven building blocks

- **API Gateway:** Managed entry point for REST/WebSocket APIs; integrates with Lambda or HTTP backends.

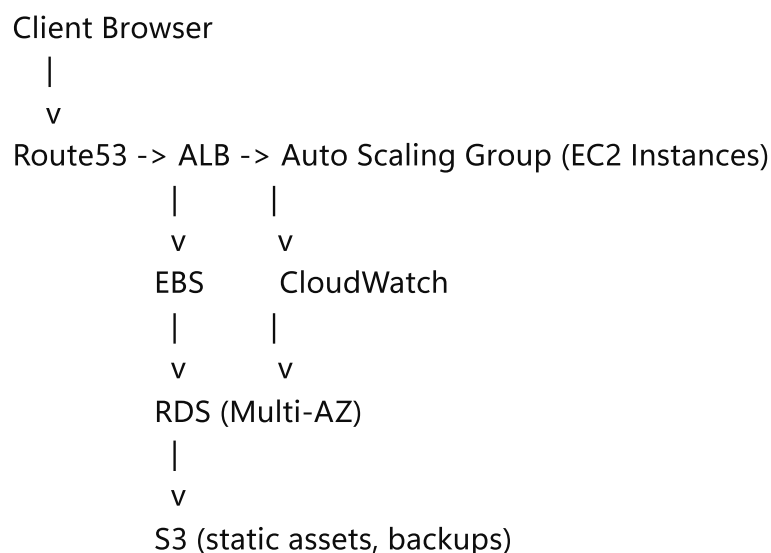
- **EventBridge:** Serverless event bus for routing and scheduling events between services and SaaS apps.
- **Step Functions:** Orchestrates serverless workflows (state machines) across Lambdas and services.

13) IoT, Mobile & Edge

- **IoT Core:** Connect and manage IoT devices securely.
- **Amplify:** Toolchain & libraries for building mobile/web apps with AWS backend (Auth, API, Storage).
- **Greengrass:** Run AWS Lambda functions and sync data on edge devices.

14) Typical architecture patterns (flowcharts)

A. Classic web app (EC2 + RDS + S3 + ALB)



Done — I created a detailed, readable note in the canvas titled **“AWS Services — Detailed Notes, Flowcharts & How They Work”** with explanations, ASCII flowcharts, use-cases and study tips.

Quick summary of the most important points (with official sources):

- AWS groups services by category (Compute, Storage, Database, Networking, etc.) and runs them across Regions and Availability Zones for reliability. [AWS Document...](#)
- **Compute:** EC2 are virtual machines you manage; Lambda is serverless function execution that auto-scales and bills per execution. [AWS Document...](#) +1
- **Storage:** S3 is object storage (buckets & objects) used for static assets and data lakes; EBS/EFS provide block and shared file storage respectively. [AWS Document...](#) +1
- **Databases:** RDS provides managed relational databases (backup/patching/Multi-AZ); DynamoDB is a managed NoSQL key-value/document DB for low-latency at scale. [AWS Document...](#) +1

Open the canvas note to read the full, long-form guide I created — it contains: categorized service descriptions, how each service works, typical clients/use-cases, three clear ASCII flowcharts (classic web app, serverless app, and a data pipeline), a decision cheat-sheet, and study/action steps.

Next steps I can do *right now* (pick any, I'll do it immediately — no waiting required):

1. Convert the ASCII flowcharts in the note into downloadable visual diagrams (PNG/SVG).
2. Produce step-by-step hands-on labs (commands + CloudFormation templates) for one or two architectures (EC2+RDS and/or Serverless).
3. Make a printable PDF of the note or a short 1-page cheat-sheet for interviews.

Which of those would you like me to do first?

 Sources

explain each service provide gcd in detail with some flowchart and how it works and there client of each one take longer and make a good note to read and understand but more detail than you gave to aws

Gcp Services — Detailed Notes, Flowcharts & How They Work

Google Cloud Platform (GCP) — Deep Reference, Flowcharts & How Each Service Works

A long, study-focused note covering GCP services in depth. Designed to be read sequentially as a reference and used for interview prep, architecture design, or hands-on learning.

Table of contents

1. Quick intro & GCP global infra
2. Compute (Compute Engine, App Engine, Cloud Run, Cloud Functions, GKE, Anthos)
3. Storage (Cloud Storage, Persistent Disk, Local SSD, Filestore, Cloud Storage for Firebase)
4. Databases (Cloud SQL, Cloud Spanner, Firestore, Bigtable, Memorystore)
5. Networking & CDN (VPC, Load Balancing, Cloud CDN, Cloud DNS, Traffic Director)
6. Security & Identity (Cloud IAM, KMS, Secret Manager, Cloud Armor, Forseti / Security Command Center)
7. Analytics & Big Data (BigQuery, Dataflow, Dataproc, Pub/Sub, Dataprep, Data Catalog)
8. Machine Learning & AI (Vertex AI, AutoML, AI infrastructure)
9. DevOps, CI/CD & Developer Tools (Cloud Build, Artifact Registry, Cloud Deploy, Cloud Source Repositories)
10. Monitoring, Tracing & Logging (Cloud Monitoring, Cloud Logging, Trace, Profiler)
11. Serverless & Event-driven building blocks (Eventarc, Cloud Tasks, Scheduler)
12. Management, Provisioning & Migration (Deployment Manager, Terraform on GCP, Migrate for Compute Engine)
13. Edge, IoT & Hybrid (Edge TPU, IoT Core history, Anthos, Bare Metal)
14. Cost, billing, quotas and pricing patterns
15. Architecture patterns & flowcharts (Classic VM app, Serverless web app, Data pipeline, Real-time analytics)
16. Decision trees: which GCP service to pick (detailed decision matrix)
17. Real-world client examples & use-cases for each service

18. Hands-on labs & migration playbook suggestions

19. Cheatsheet & interview bullet points

1) Quick intro & GCP global infrastructure

What is GCP? Google Cloud Platform (GCP) is Google's suite of public cloud computing services. It offers compute, storage, networking, analytics, machine learning, identity, and developer tooling. GCP runs on Google's global infrastructure: regions (geographic areas) composed of zones (isolated failure domains). Use zones for high-availability within a region and multi-region for resilience across regions.

How Google positions GCP:

- Strong in data, analytics and machine learning (BigQuery, TPU/AI tooling).
 - Provides managed services that integrate tightly with Google's internal systems and networking.
-

2) Compute — options, how they differ, when to pick each

GCP provides a spectrum of compute: fully managed PaaS, serverless, container platforms and raw VMs. Each has a clear trade-off between control and operational overhead.

Compute Engine (VMs / IaaS)

- **What:** Raw virtual machines (VMs) — select machine type, CPU, memory, boot disk. Supports custom machine types and both Linux and Windows.
- **How it works:** Create instances in a VPC subnet. Attach Persistent Disks (block storage) for durability. Use Instance Groups for horizontal scaling; Managed Instance Groups (MIGs) support autohealing and autoscaling.
- **Billing model:** Charged per vCPU/GB-second of VM runtime; sustained/committed use discounts available. Persistent disks billed separately.
- **Best for:** Lift-and-shift migrations, legacy apps, stateful services, workloads needing full OS control.

- **Client examples:** Enterprises migrating on-prem apps, specialized software needing kernel-level tuning.

App Engine (PaaS)

- **What:** Platform-as-a-Service for web apps. Two environments: Standard (sandboxed with language runtimes and fast scaling) and Flexible (runs containers on Compute Engine, more control).
- **How it works:** Deploy application code. Platform handles provisioning, load balancing, scaling and patching. Standard environment uses predefined runtimes; Flexible uses Docker.
- **Billing:** Per instance/instance-hour for flex, per request + instance for standard variants.
- **Best for:** Rapidly deploying web apps without infra management, apps with predictable frameworks.
- **Clients:** SaaS startups, web applications with minimal infra ops.

Cloud Run (serverless containers)

- **What:** Fully managed serverless platform for stateless containers (based on Knative). Also available as Cloud Run for Anthos for GKE.
- **How it works:** Build/deploy container images. Cloud Run scales from zero to N based on requests, and bills per request duration and memory/CPU allocated.
- **Why pick it:** If you want serverless scaling while packaging code as containers (bring your own runtime/libraries).
- **Clients:** Microservices, APIs, event-driven backends.

Cloud Functions (FaaS)

- **What:** Event-driven functions (single-purpose) triggered by HTTP, Pub/Sub, Cloud Storage, etc.
- **How it works:** Deploy functions in supported languages; GCP provisions execution environment on demand and charges per invocations and compute time.
- **Best for:** Lightweight event processing, file transform pipelines, webhook handlers.

Google Kubernetes Engine (GKE)

- **What:** Managed Kubernetes offering. You can run Kubernetes clusters with control-plane management by Google.
- **How it works:** GKE manages control plane upgrades, while worker nodes are Compute Engine VMs or GKE Autopilot nodes (more opinionated, managed).
- **Why pick GKE:** When you need container orchestration, multi-container microservices, portability across clouds.
- **Clients:** Organizations standardizing on Kubernetes.

Anthos (Hybrid & multi-cloud)

- **What:** Platform to run GKE-based workloads across on-prem, other clouds, and GCP.
 - **How it works:** Anthos provides management/control plane and Policy Controller for hybrid deployments, enabling consistent clusters across environments.
 - **Clients:** Enterprises with hybrid/multi-cloud strategy.
-

3) Storage — types, tradeoffs, and how they function

GCP offers object, block, file and archival storage. Choose by access pattern, latency and consistency needs.

Cloud Storage (object)

- **What:** Global, highly durable object storage (buckets/objects) used for static content, backups, and data lakes.
- **How it works:** Data stored as objects with metadata. Storage classes (Standard, Nearline, Coldline, Archive) for cost/latency tradeoffs. Integrates with IAM, signed URLs, lifecycle rules, and nearline archival.
- **Use-cases:** Static website hosting, media storage, data lake for analytics.

Persistent Disk (block storage)

- **What:** Block-level storage attachable to Compute Engine VMs. Options include SSD and balanced PDs, and Hyperdisk for high-performance needs.
- **How it works:** Provision disks and attach to VMs; persistent across reboots. Snapshots for backups.

- **Use-cases:** Databases on VMs, boot disks.

Local SSD

- **What:** Ephemeral, NVMe-like attached storage with very low latency and high IOPS, but data lost when VM stops.
- **Use-cases:** Caching, temp storage for high-throughput workloads.

Filestore (managed NFS)

- **What:** Managed file storage (NFS) for applications requiring POSIX semantics and shared mounts across VMs/K8s.
- **Use-cases:** Content management systems, media processing workflows.

Cloud Storage for Firebase

- **What:** Built on Cloud Storage but tailored to mobile/web apps with client SDKs and security rules.

4) Databases — transactional, NoSQL, wide-column and in-memory

GCP has multiple database services aimed at different needs. The decision depends on consistency, scale, schema, latency and query model.

Cloud SQL (managed RDBMS)

- **What:** Managed MySQL, PostgreSQL, SQL Server instances.
- **How it works:** Google manages patching, backups, replication, and failover; offers read replicas.
- **Use-cases:** Traditional OLTP web backends, applications using relational schema.

Cloud Spanner (globally-distributed RDBMS)

- **What:** Horizontally scalable relational DB with strong consistency and SQL support.
- **How it works:** Synchronous replication across zones/regions, distributed transactions with external consistency.

- **Use-cases:** Global transactional applications (financial systems, global inventory, gaming leaderboards needing global consistency).

Firestore (document DB) & Firebase Realtime Database

- **What:** Serverless NoSQL document DB with real-time sync capabilities. Firestore offers richer querying and stronger consistency semantics compared to Realtime DB.
- **Use-cases:** Mobile/web apps needing real-time updates and offline-first behavior.

Bigtable (wide-column)

- **What:** Low-latency, high-throughput wide-column datastore — ideal for analytical and operational workloads like time-series and ad-tech.
- **How it works:** Single-key read/writes at massive scale; not designed for complex multi-row transactions.
- **Use-cases:** Time-series, IoT, personalization engines.

Memorystore (Redis / Memcached)

- **What:** Managed in-memory caching service for low-latency access.
- **Use-cases:** Session stores, leaderboard caching, caching DB query results.

BigQuery (analytic data warehouse)

- **What:** Serverless, highly scalable, columnar data warehouse with SQL interface. Separates storage and compute; supports petabyte-scale datasets.
- **How it works:** Query data stored in tables (or external tables on Cloud Storage) using ANSI SQL-like syntax. Supports streaming inserts and batch loads, ML with BigQuery ML.
- **Use-cases:** BI and analytics, ad-hoc SQL, data science.

5) Networking & CDN

VPC (Virtual Private Cloud)

- **What:** Global virtual network with subnets in regions. Supports Shared VPC, VPC peering, and Private Google Access.

- **Key features:** Firewall rules, routes, Cloud NAT for outbound internet from private VMs.

Cloud Load Balancing

- **What:** Global L7 (HTTP(S)) and regional L4 load balancers, with cross-region failover.
- **How it works:** Uses Google's global front-end and anycast IPs for traffic; integrates with backend services (instance groups, serverless backends).

Cloud CDN

- **What:** CDN that caches origins (Cloud Storage, Cloud Run, GCE) at edge locations.

Cloud DNS

- **What:** Scalable, managed DNS service.

Traffic Director

- **What:** Managed service mesh control plane for global traffic management across Envoy proxies (used with GKE/Anthos).

6) Security & Identity

Cloud IAM

- **What:** Centralized role- and policy-based access control for resources.
- **Best practice:** Use principle of least privilege and use service accounts for applications.

KMS (Key Management Service)

- **What:** Create and manage encryption keys; integrate with Cloud Storage

Done — I created a long, detailed note in the canvas titled **"GCP Services — Detailed Notes, Flowcharts & How They Work"** covering compute, storage, databases, networking, security, analytics, ML, serverless, hybrid, pricing patterns, decision trees, and hands-on playbooks.

Quick highlights and the most important facts (with sources):

- Google Cloud lists 100+ products across compute, storage, AI/ML, analytics and more — the Products page is the canonical catalog. [Google Cloud](#)
- For compute, Cloud Run, App Engine and Cloud Functions are serverless billing-per-request options, while Compute Engine and GKE are resource-based (VMs) with different cost models and trade-offs. [Google Cloud +1](#)
- Storage choices fall into object (Cloud Storage), block (Persistent Disk / Hyperdisk), and file (Filestore); choose based on latency, persistence and POSIX needs. [Google Cloud +1](#)
- Databases: Cloud SQL for managed relational needs; Cloud Spanner when you need global transactional consistency at scale; Firestore/Bigtable for NoSQL/document/wide-column patterns; BigQuery for serverless analytics. [Google Cloud +2](#)

What I can do next — I'll do it immediately if you pick one (no waiting):

1. Convert the ASCII flowcharts into downloadable PNG/SVG diagrams.
2. Generate a printable PDF of the full note.
3. Produce two hands-on labs (with commands and Terraform/Deployment Manager templates): (A) Classic VM web app migration to Compute Engine + Cloud SQL; (B) Serverless app on Cloud Run + Firestore + Cloud Build.
4. Produce a 1-page cheat-sheet + 20 interview flashcards from the note.

Which one would you like first?

 Sources